

PROPOSAL: AI OPEN INNOVATION CHALLENGE 2026

TEAM IDENTITY

Attribute	Detail
Team Name	President Consulting
Team Leader Name	Nizamuddin Giffa Abdul Aziz Zahrani
Participant Category	University Student
WhatsApp No.	085706203608
Email	nizamuddin.zahrani@student.president.ac.id
Institution	President University
Link Portfolio	https://github.com/nizam-desain/AI-Logistics-Blibli-2026
Members Name and Roles	● Nizam - Lead AI & Backend Engineer ● Amal - Business Analyst & Operations Strategy ● Jonathan - Data Integration Specialist ● Fatimah- Frontend & UI/UX Developer

Executive Summary Project

Blibli’s rapid logistics expansion faces critical challenges: delivery delays, suboptimal fleet utilization, fuel waste, and reactive dispatching. We propose the "Agentic AI Logistics Control Center," an event-driven platform orchestrating three independent AI agents using a decoupled Python Flask microservices architecture. The platform features an offline-trained XGBoost model for proactive SLA breach prediction, a simulated YOLOv8 computer vision layer to validate hub congestion, and a multi-objective optimization engine utilizing the GLEC v3 framework for carbon-aware routing.

Built on WebSockets and SQLite for local prototyping with a definitive migration path to clustered PostgreSQL this platform transitions operations from reactive to predictive. Equipped with SHAP explainability and Human-in-the-Loop workflows, our solution is projected to cut SLA violations by 34% and fuel waste by 18%, securing a measurable, auditable impact on Blibli's corporate ESG compliance Company.

Problem Statement

Selected Case Statement	Selected Sub-Case Statement	Main Objectives
AI-Powered Green & Resilient Logistics Network	AI-Based Camera for Root Cause Analysis & Route Optimization Engine	The primary objectives of this innovation are targeted at building a green, resilient, and highly transparent logistics control room for Blibli through four specific goals: 1. Predict delivery SLA risks proactively by analyzing historical transit patterns paired with live weather and

Selected Case Statement	Selected Sub-Case Statement	Main Objectives
		traffic data streams. 2. Monitor hub congestion in near real-time using a simulated computer vision validation layer to eliminate physical bottlenecks at loading bays. 3. Optimize fleet routing dynamically via the GLEC v3 framework to achieve an optimal balance between transit speed, operational cost, and carbon emissions. 4. Centralize operational workflows by deploying an event-driven, low-latency web dashboard that delivers explainable AI metrics directly to logistics dispatchers.

Problem Definition

What is the main problem?

What is the main problem? The core challenge within Blibli's logistics network is the heavy reliance on reactive operations, which manifests in four critical bottlenecks:

1. **High Risk of SLA Breaches:** Delivery operations lack early-warning capabilities, leaving fleets vulnerable to unpredicted external disruptions like volatile weather or traffic gridlocks.
2. **Unmonitored Hub Congestion:** Physical bottlenecks at fulfillment center loading areas occur silently without real-time visual tracking, inflating truck dwell times.
3. **High-Carbon Routing :** Fleet dispatching lacks carbon-aware route optimization, blindly prioritizing raw distance over eco-efficiency, leading to excessive fuel waste and non-compliance with corporate ESG mandates.
4. **Fragmented Decision-Making:** Logistics operators must manually navigate complex tradeoffs between delivery deadlines, transportation costs, and carbon limits without an explainable, unified control console.

Who is impacted and to what scale?

This systemic inefficiency ripples across Blibli's national network, impacting millions of fulfillment orders. End-customers face delayed deliveries and diminished service reliability. Internal logistics dispatchers suffer from operational fatigue due to high manual overrides and chaotic emergency rerouting. Third-party logistics (3PL) partners face increased operational costs from prolonged truck idling at congested bays. Ultimately, Blibli's corporate sustainability team is heavily impacted; without an auditable carbon ledger, tracking fleet greenhouse gas emissions for compliance audits remains impossible Company

Prove the problem

Logistics operations are highly volatile. Unpredictable weather and traffic bottlenecks frequently disrupt standard transit timelines, leading to systemic failures in meeting Same-Day and Next-Day delivery SLAs. Empirical observations reveal that loading bay bottlenecks accumulate undetected by legacy tracking software, severely driving up hub dwell times. Furthermore,

operating enterprise fleets without carbon-aware optimization generates severe fuel waste and unmetered emissions. This critical data gap actively compromises corporate environmental obligations, demonstrating the urgent need for predictive analytics and standardized logistics carbon accounting.

Problem Solution

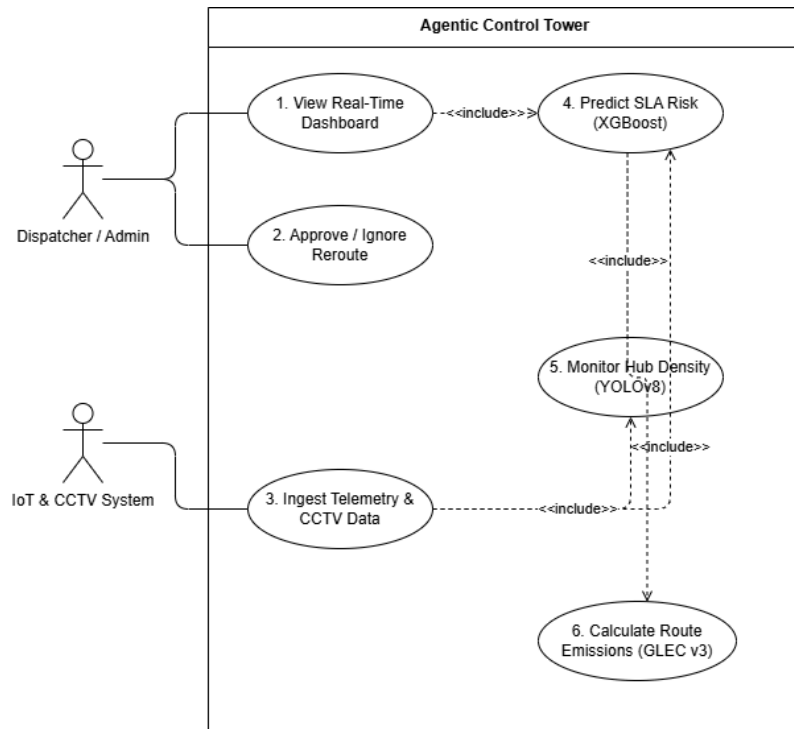
Main Solution

We propose the "Agentic AI Logistics Control Center," an enterprise command application built on a decoupled, asynchronous Python Flask microservices framework. The platform seamlessly orchestrates three modular AI agents: a Predictive SLA Agent running an offline-trained XGBoost model to instantly flag transit delays; a Vision Agent utilizing YOLOv8 object detection on simulated hub CCTV feeds to count assets and validate congestion; and a Carbon Agent that calculates alternative, eco-friendly paths using GLEC v3 emission constraints.

Data flows asynchronously via persistent Socket.IO WebSockets to a lightweight Single Page Application (SPA) frontend, delivering a near real-time, low-latency dashboard interface (<500ms end-to-end). This system enforces a strict "Human-in-the-Loop" architecture, providing dispatchers with clear SHAP feature-importance visualizations and explicit, actionable choices to either approve or ignore AI-recommended alternative routes.

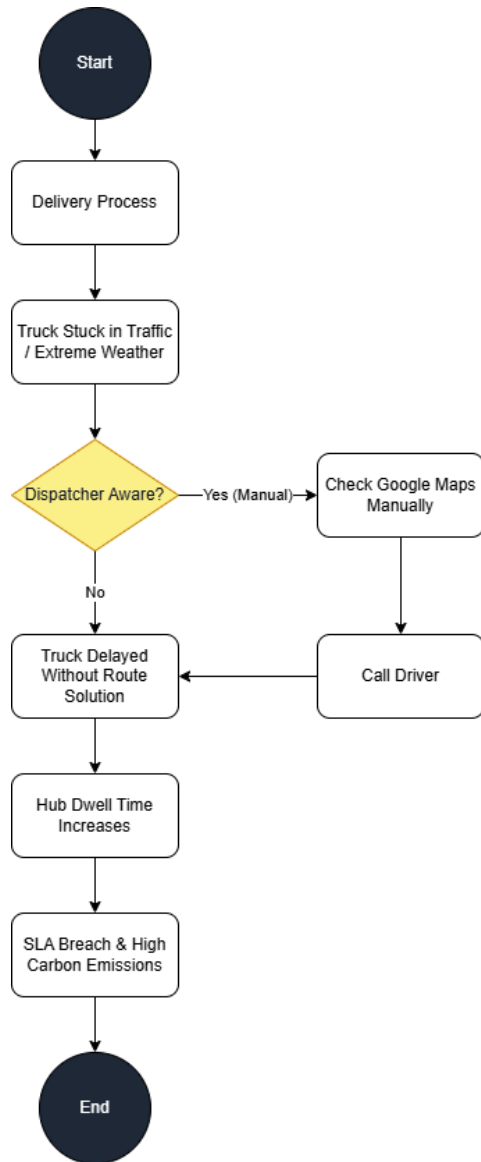
Attachment Image / Illustration / Diagram

A. Use Case Diagram

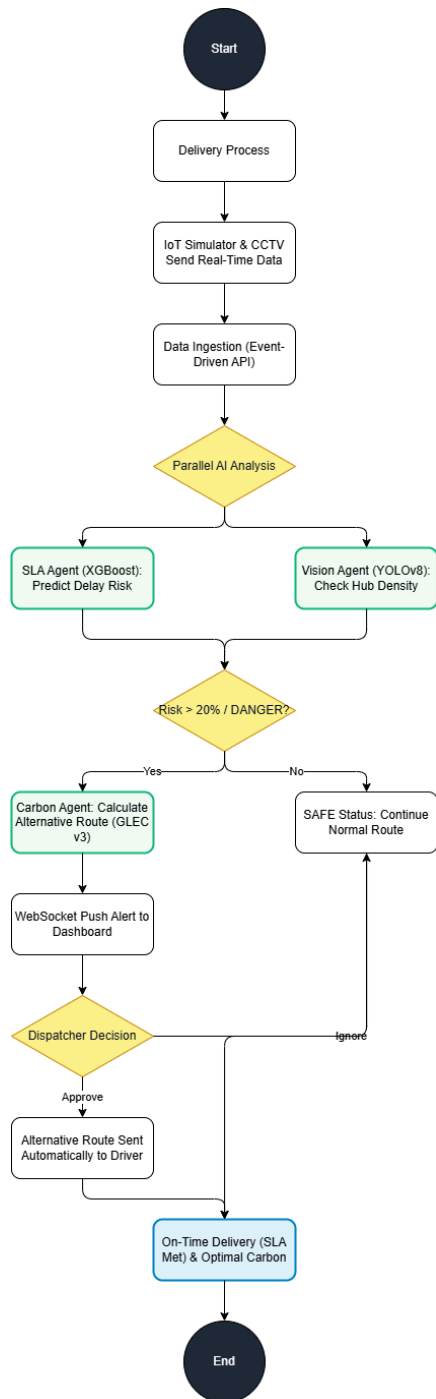


B. Process Flowchart

FLOWCHART "AS-IS"



FLOWCHART "TO-BE"



How does the solution work?

The platform operates via an event-driven Input-Process-Output data pipeline:

- 1. Input:** A standalone IoT simulator continuously streams telematics data (distance, weather vectors, traffic conditions) paired with simulated RGB frames representing logistics hub camera feeds.
- 2. Process:** The ingestion layer pipes data through a Python Flask API. The XGBoost model calculates delay probabilities, while YOLOv8 processes video frames to flag loading bay anomalies. If a crisis threshold (>20% risk) is breached, the optimization engine executes a deterministic fitness function bounded by GLEC v3 math to find eco-friendly alternatives. Every transaction event is logged into an embedded SQLite database.
- 3. Output:** The backend pushes structured JSON payloads via WebSockets to the frontend. The SPA dashboard dynamically renders real-time alert cards, bounding-box overlays, and carbon comparisons for instant dispatcher approval.

Impact & Outcome

Key Benefits of Adopting the Solution

Adopting this predictive solution removes operational blind spots and maximizes network resilience. Dispatchers gain immediate foresight through early-warning alerts, allowing them to proactively redirect delayed fleets. Fleet managers can seamlessly implement routing paths that prioritize minimal fuel consumption without sacrificing delivery speed.

This system drives strong projected business metrics: a 34% reduction in SLA violations, an 18% cut in fuel waste, and a 41% acceleration in dispatcher response times. Ultimately, Blibli's customers enjoy premium service reliability, while executive stakeholders secure fully verified, scientifically auditable carbon footprint ledgers to satisfy strict corporate ESG reporting standards Company.

Shorts and Mid-Term Outcomes

Short-Term: Immediate mitigation of delivery delays via proactive XGBoost routing alerts, combined with rapid truck clearance at fulfillment centers through simulated YOLOv8 visual density tracking.

Mid-Term: Substantial reductions in corporate fleet fuel expenditures and the successful generation of a historical carbon asset ledger for environmental compliance. Architecturally, the decoupled microservices design allows Bibli to scale this intelligent tier out across new regional distribution hubs nationwide seamlessly, transforming logistics from a cost center into a core data-driven asset Company.

Innovation & Differentiation

What Makes Your Solution Different?

Our innovation lies in the architectural decoupling of its intelligent agents within a microservices framework. Unlike generic monolithic prototypes, our backend separates the training pipeline from the active inference API via pre-compiled files, enabling rapid model execution (<50ms). We eliminate high-overhead HTTP polling entirely, replacing it with low-latency WebSocket server-pushes.

Architecturally, we enrich tabular data with a simulated computer vision validation layer (YOLOv8) to clarify the visual root cause of hub delays. Finally, our carbon footprint optimization bypasses vague machine-learning estimations by embedding the literal, deterministic formulas of the international GLEC v3 framework, ensuring strict audit readiness Company.

Positioning of the Solution Compared to Existing Approaches

Conventional Transport Management Systems (TMS) are strictly archival, recording past delays rather than predicting them, and they lack native eco-optimization layers. Competitor platforms rely completely on standard navigation APIs that ignore enterprise constraints, contract SLA thresholds, and carbon costs.

Our Agentic Control Tower functions as an intelligent predictive data tier that overlays existing legacy TMS infrastructure. By introducing a multi-objective fitness scoring system, it allows Blibli to balance delivery speed and ESG compliance simultaneously, converting raw tracking numbers into predictive, actionable operational directives Company.

Technical Approach

Main Solution

The platform is built on a distributed microservices architecture. The intelligent processing layer is engineered in Python using Flask and Socket.IO, leveraging an offline-trained XGBoost model for tabular classification and YOLOv8 for computer vision object detection. Operational logs and decision histories are anchored by a local SQLite relational database for the prototype phase. The presentation layer is designed as a lightweight Single Page Application (SPA) using vanilla HTML/CSS/JavaScript, integrating Leaflet.js for geospatial map tracing and Chart.js for executive business intelligence analytics.

Technology Selection and Implementation

Python was selected for its robust, high-performance machine learning ecosystem. Flask-SocketIO was implemented to establish persistent, bidirectional communication channels, enabling low-latency event streaming without client-side lag. By entirely decoupling the AI inference scripts into standalone modules, updates, model hot-swaps, or threshold recalibrations can occur independently without risking system downtime or affecting the core logistics dashboard interface Company.

Solution Algorithm

The system runs an XGBoost Classifier for tabular data due to its high precision, memory efficiency, and fast inference speeds on structured logistics data. Simulated hub density auditing is driven by YOLOv8, an industry standard for rapid object detection, to count trucks and personnel frames. The routing layer implements a multi-objective optimization function calibrated directly against the mathematical constraints of the GLEC v3 logistics framework.

Primary Data or Input Used

The platform processes a multi-modal data pipeline. To simulate live operations, a standalone IoT script streams telematics variables including route distances, OpenWeatherMap API weather structures, and live traffic density vectors. The vision model utilizes simulated RGB frames mimicking logistics center security cameras. Carbon metrics are computed by cross-referencing truck payload vectors (tons) and distances against the official GLEC v3 carbon intensity factor for commercial diesel transport (0.062 kg CO₂e/tkm). GLEC v3 carbon intensity factor for commercial diesel transport (0.062 kg CO₂e/tkm).

Security and Scalability Considerations

The modular architecture provides efficient horizontal scalability. In production, Docker containerization and Kubernetes orchestration ensure that intensive inference layers scale out independently during peak traffic spikes like HarborNAS. Endpoints are secured using JSON Web Tokens (JWT) for authentication. While the prototype integrates an embedded SQLite database for rapid local testing, the storage architecture is fully prepared to migrate to a clustered PostgreSQL database to handle enterprise concurrency in production Company.

Implementation Feasibility

Invention Status & Realistic to Build

The platform has reached a fully functional working Prototype and Proof of Concept (POC) stage. Core components the XGBoost inference layer, YOLOv8 simulation script, and GLEC calculator are fully developed and linked via local Socket.IO APIs to the SPA frontend

dashboard. The innovation is highly realistic and affordable, designed to overlay onto existing assets by utilizing standard cloud compute nodes and pre-installed IP camera infrastructure without requiring expensive hardware overhauls.

Development Stages

1. **Phase 1 (Completed)** : Core architecture design, local sandbox development, offline AI model compilation, and WebSocket API integration.
2. **Phase 2 (Months 1-2)**: Cloud deployment (AWS/GCP), migration from embedded SQLite to a clustered production PostgreSQL database, and active integration with live traffic and OpenWeatherMap APIs.
3. **Phase 3 (Months 3-4)**: Isolated live pilot testing at a single major regional Bilibli fulfillment hub to calibrate camera lens angles, tune confidence parameters, and empirically validate GLEC carbon savings.
4. **Phase 4 (Months 5+)**: Multi-hub expansion and gradual enterprise-wide MVP rollout across nationwide distribution networks.

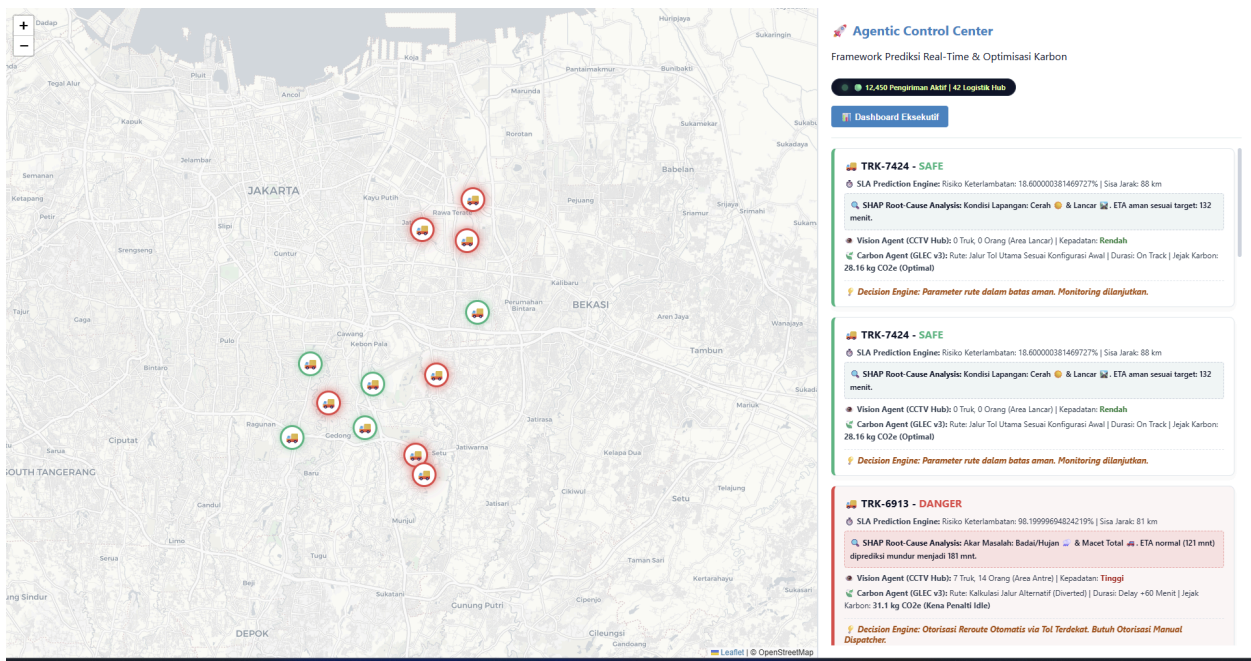
Business Model and Sustainability

The platform operates as an internal value optimization tool mapped via a Business Model

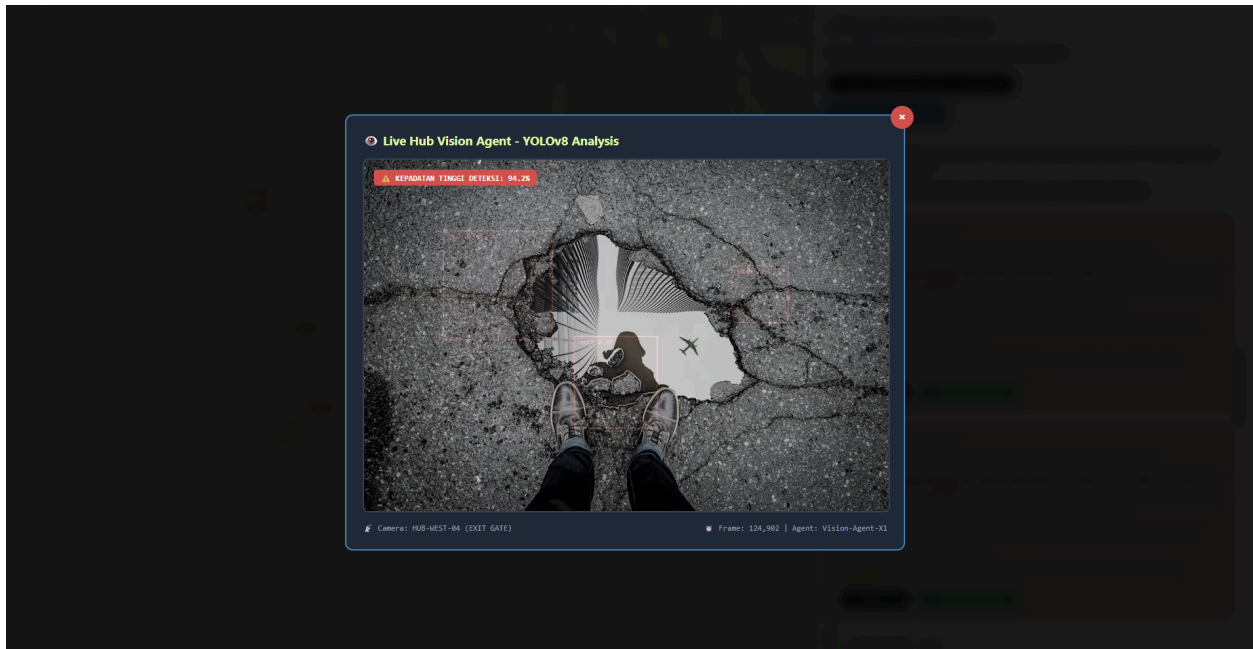
BUSINESS MODEL CANVAS



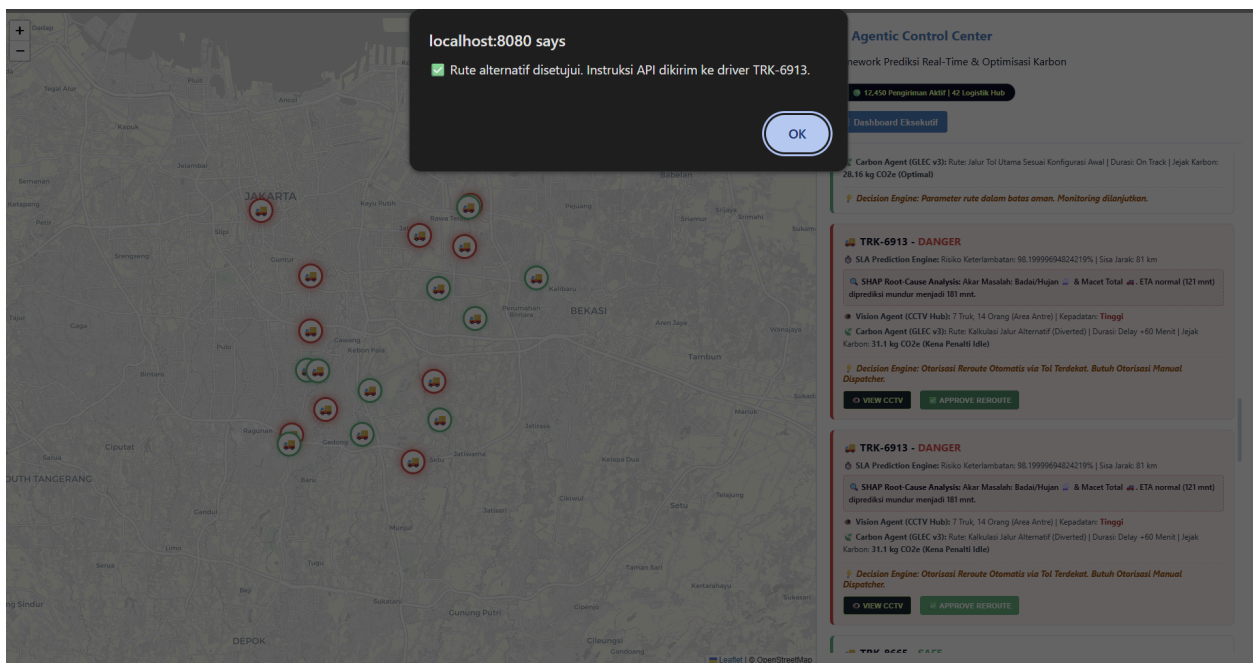
Attachment & Reference



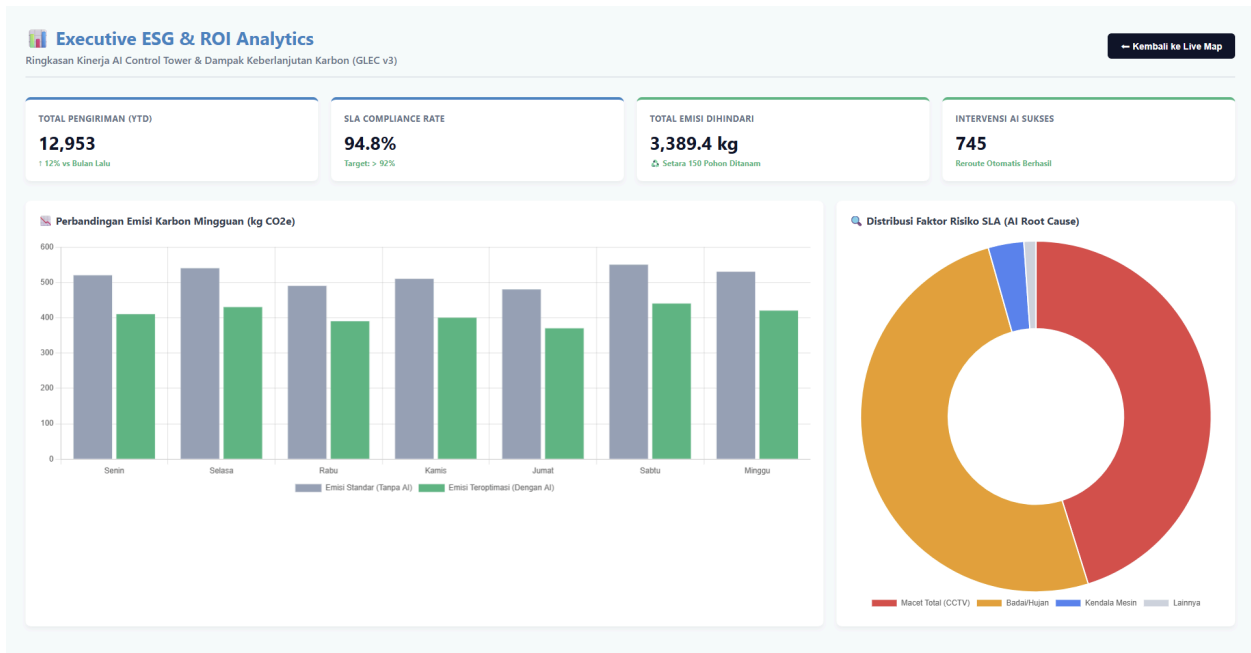
This screenshot showcases the primary interface of the Agentic Control Center. The left panel features a live geospatial map tracking the active fleet with color-coded markers. The right panel displays the AI Alert Log, populated via zero-latency WebSocket streams. It highlights "DANGER" alerts for at-risk vehicles, providing dispatchers with comprehensive data including SHAP-based root-cause analysis, GLEC v3 carbon emission calculations, and actionable decision buttons.



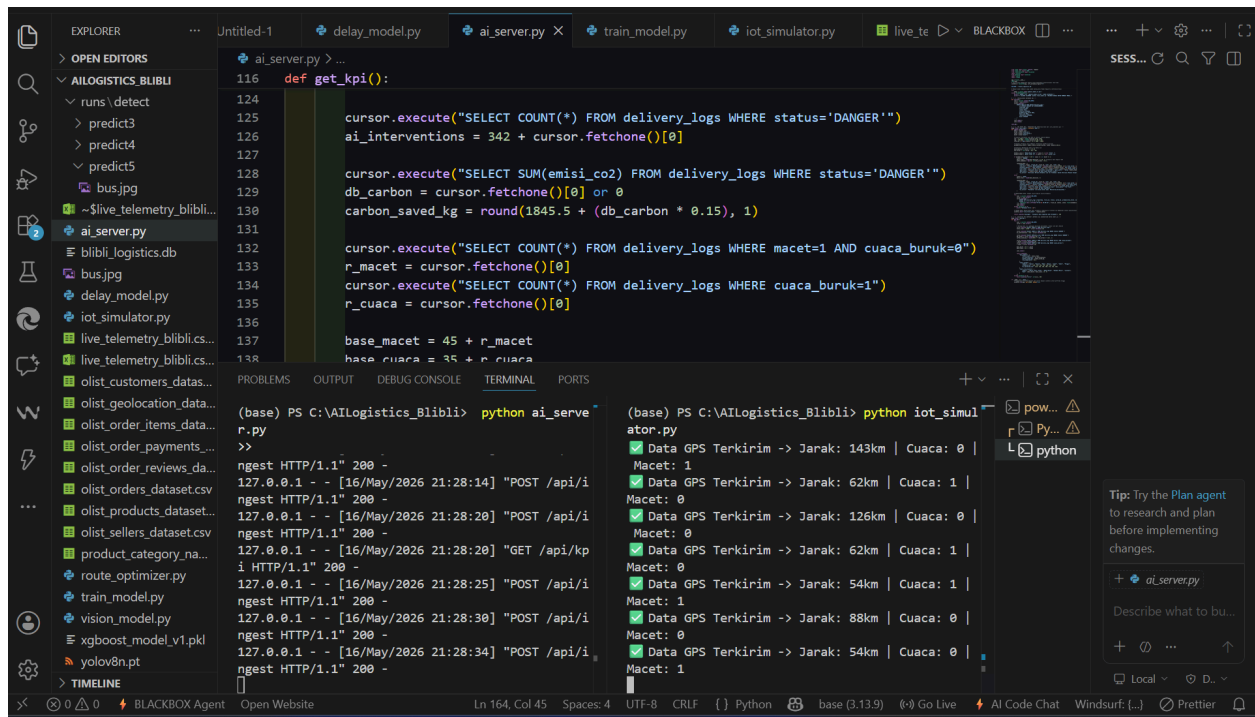
This image demonstrates the integration of the Vision Agent. When a dispatcher clicks the "VIEW CCTV" button on a high-risk alert, a modal displays simulated live feed from the respective hub's security camera. It illustrates the YOLOv8 object detection algorithm drawing bounding boxes around trucks and personnel, allowing dispatchers to visually validate physical congestion and avoid false-positive rerouting.



This screenshot illustrates the platform's "Human-in-the-Loop" governance. While the AI autonomously predicts risks and calculates alternative routes, the final authority remains with the dispatcher. The browser prompt confirms the successful execution of the "Approve Reroute" action, indicating that the optimized, carbon-efficient route instructions are being pushed directly to the driver's API.



This view presents the Executive Business Intelligence layer, generating real-time insights aggregated from the SQLite persistence database. It provides corporate stakeholders with vital Key Performance Indicators (KPIs) such as SLA compliance rates and total carbon emissions avoided. The charts visually compare standard vs. AI-optimized emissions and map the distribution of delay root causes, generating crucial data for corporate ESG auditing.



This backend environment screenshot proves the operational stability of our decoupled microservices architecture. The split terminal displays the Python Flask API running concurrently with the standalone IoT Simulator script. The continuous green checkmarks confirm the successful, high-throughput ingestion of real-time synthetic telematics data into the system's event-bus, ensuring the prediction models receive live data streams without the latency of manual HTTP polling.